

Assignment 2 extra

Hadeer Ghoneim

Verification Diploma

Question1

```
1  module data_types_extra;
2
3      // 1) Declare a 2-state array that holds four 12-bit values
4      bit [11:0] my_array [0:3];
5
6      initial begin
7          // 2) Initialize array
8          my_array[0] = 12'h012;
9          my_array[1] = 12'h345;
10         my_array[2] = 12'h678;
11         my_array[3] = 12'h9AB;
12
13         // Print header
14         $display("---- Using for loop ----");
15
16         // 3a) Traverse using for loop
17         for (int i = 0; i < 4; i++) begin
18             $display("my_array[%0d][5:4] = %b", i, my_array[i][5:4]);
19         end
20
21         $display("---- Using foreach loop ----");
22
23         // 3b) Traverse using foreach loop
24         foreach (my_array[i]) begin
25             $display("my_array[%0d][5:4] = %b", i, my_array[i][5:4]);
26         end
27     end
28
29 endmodule
30
```

```

# ---- Using for loop ----
# my_array[0][5:4] = 01
# my_array[1][5:4] = 00
# my_array[2][5:4] = 11
# my_array[3][5:4] = 10
# ---- Using foreach loop ----
# my_array[0][5:4] = 01
# my_array[1][5:4] = 00
# my_array[2][5:4] = 11
# my_array[3][5:4] = 10

```

Question 2

1. Testbench code

```

1  module ALU_tb;
2
3      import ALU_pkg::*;
4
5      // =====
6      // DUT signals
7      // =====
8      logic clk;
9      logic reset;
10     opcode_t Opcode;
11     logic signed [3:0] A, B;
12     logic signed [4:0] C_dut;
13
14     // =====
15     // DUT instantiation
16     // =====
17     ALU dut (
18         .clk    (clk),
19         .reset   (reset),
20         .Opcode  (Opcode),
21         .A       (A),
22         .B       (B),
23         .C       (C_dut)
24     );
25
26     // =====
27     // Transaction object
28     // =====
29     ALU_inputs tr;
30

```

```

31 // =====
32 // Golden model is pure combinational
33 // =====
34 function automatic logic signed [4:0] golden_model(
35     input opcode_t op,
36     input logic signed [3:0] a,
37     input logic signed [3:0] b,
38     input logic rst
39 );
40     if (rst) begin
41         golden_model = '0;
42     end else begin
43         case (op)
44             ADD:         golden_model = a + b;
45             SUB:         golden_model = a - b;
46             NOT_A:       golden_model = ~a;
47             REDUCTION_OR: golden_model = |b;
48         endcase
49     end
50 endfunction
51
52 // =====
53 // Clock generation
54 // =====
55 always #5 clk = ~clk;
56
57 // =====
58 // Self-check block
59 // =====
60 int pass_count = 0;
61 int err_count  = 0;
62

```

```

63 initial begin
64     // Init
65     clk      = 0;
66     reset    = 1;
67     Opcode   = ADD;
68     A        = '0;
69     B        = '0;
70
71     // Hold reset a few cycles
72     repeat(2) @(posedge clk);
73     reset = 0;
74
75     // Create transaction object
76     tr = new();
77
78     // Run randomized test
79     repeat (200) begin
80
81         logic signed [4:0] expected;
82
83         // Randomize transaction
84         assert(tr.randomize()) else $fatal("Randomization failed!");
85
86         // Drive signals
87         A      = tr.A;
88         B      = tr.B;
89         Opcode = tr.Opcode;
90         reset  = tr.reset;
91
92         // Wait for clock edge so DUT actually registers outputs
93         @(posedge clk);
94
95         // Golden model expected value
96         expected = golden_model(Opcode, A, B, reset);
97

```

```

97
98     // Compare
99     if (C_dut != expected) begin
100         $display("ERROR: op=%s A=%0d B=%0d -> DUT=%0d, EXP=%0d, reset=%0b",
101             Opcode.name(), A, B, C_dut, expected, reset);
102         err_count++;
103     end else begin
104         pass_count++;
105     end
106
107     // Golden model expected value
108     expected = golden_model(Opcode, A, B, reset);
109
110 end
111
112 $display("✅ Simulation finished: pass=%0d, errors=%0d", pass_count, err_count);
113 $stop;
114 end
115
116 endmodule
117

```

2. Package code

```

1 package ALU_pkg;
2 // -----
3 // Typedef enum for opcode
4 // -----
5 typedef enum logic [1:0] {
6     ADD          = 2'b00,
7     SUB          = 2'b01,
8     NOT_A        = 2'b10,
9     REDUCTION_OR = 2'b11
10 } opcode_t;
11 // -----
12 // ALU Transaction Class
13 // -----
14 class ALU_inputs;
15     rand bit signed [3:0] A;
16     rand bit signed [3:0] B;
17     rand opcode_t Opcode;
18     rand bit reset;
19
20     // Constraint: reset is low most of the time
21     constraint c_reset {
22         reset dist { 1 := 1, 0 := 9 }; // 10% high, 90% low
23     }
24
25     // Constructor
26     function new();
27     endfunction
28
29     // For debugging / printing
30     function void display();
31         $display("Time=%0t | reset=%0b | Opcode=%s | A=%0d | B=%0d",
32             $time, reset, Opcode.name(), A, B);
33     endfunction
34 endclass
35
36 endpackage

```

3. Design code

```
1  module ALU (  
2      input  clk,  
3      input  reset,  
4      input  [1:0] Opcode,    // The opcode  
5      input  signed [3:0] A,  // Input data A in 2's complement  
6      input  signed [3:0] B,  // Input data B in 2's complement  
7      output reg signed [4:0] C // ALU output in 2's complement  
8  );  
9      reg signed [4:0] Alu_out; // ALU output in 2's complement  
10     localparam      Add      = 2'b00; // A + B  
11     localparam      Sub      = 2'b01; // A - B  
12     localparam      Not_A    = 2'b10; // ~A  
13     localparam      ReductionOR_B = 2'b11; // |B  
14     // Do the operation  
15     //covered all cases  
16     always @* begin  
17         case (Opcode)  
18             Add:      Alu_out = A + B;  
19             Sub:      Alu_out = A - B;  
20             Not_A:    Alu_out = ~A;  
21             ReductionOR_B: Alu_out = |B;  
22         endcase  
23     end // always @ *  
24     // Register output C  
25     // why do not remove c ??  
26     // why use internal signal alu_out!!!?  
27     //maybe to avoid latches???!!  
28     //you want alu with register?  
29     always @(posedge clk or posedge reset) begin  
30         if (reset)  
31             C <= 5'b0;  
32         else  
33             C <= Alu_out;  
34     end  
35  
36 endmodule
```

5. Snippet to your verification plan document

Label	Design requirement description	Stimulus generation	Functional coverage (suggested coverpoints / bins)	Functionality check (self-checking)
ALU_RESET	When reset is asserted (1) the output C must be 0 (synchronous asynchronous as RTL).	Directed: initial reset=1 for several clock cycles, then deassert. Also randomize reset (constrained low most of time) during run to exercise both cases.	Coverpoint: reset { bins: asserted, deasserted }. Cross reset with time/clock edges to ensure both sampled and asynchronous cases.	Checker: After each @(posedge clk) if reset==1 then expect C==0. Report error if not.
ALU_ADD_BASIC	Opcode = ADD performs signed addition: $C = A + B$ (5-bit signed result).	Constrained random: random A/B across range plus directed edge cases (max pos 7, max neg -8, zero).	Coverpoints: Opcode==ADD; A bins {neg, zero, pos, max_pos, max_neg}; B same; cross Opcode * A * B (ensure combos). Goal: 100% of coverpoints.	Checker: golden-model compute expected = A + B and compare to registered C after clock. Log mismatch: values and expected.
ALU_SUB_BASIC	Opcode = SUB performs signed subtraction: $C = A - B$.	Same as ADD: constrained random + directed corner cases.	Coverpoints: Opcode==SUB; cover bins for A/B. Cross Opcode * sign(A) * cover(A) * cover(B).	Checker: expected = A - B. Compare and log errors.
ALU_NOTA	Opcode = NOT_A performs bitwise inversion on A ($C = \sim A$).	Randomize A; include directed vectors of 0, +1, -1, max values.	Coverpoints: Opcode==NOT_A; A bins to ensure all bit positions toggled (e.g., include values that exercise msb, lsb).	Checker: expected = ~A (assign to 5-bit signed as in RTL). Compare to C.
ALU_REDUCTION_OR	Opcode = REDUCTION_OR produces "C" =	B' (reduction OR of B -> 0 or 1).	Randomize B; ensure some vectors with zero and non-zero. Coverpoint: clk edges observed; cross reset with clk	Coverpoints: Opcode==REDUCTION_OR; B bins {all_zero, any_one}.
ALU_REGISTER_UPDATE	Output C updates only on posedge clk and obeys reset semantics (register stage exercised).	Directed + random: toggle clk, apply inputs around edges; assert/deassert reset at different times.	to ensure both branches executed. Coverage of the if (reset)/else branches in the register always happens.	Checker: Use registered copies of inputs (A_reg, B_reg, Opcode_reg) captured on posedge clk and compute golden result from these. After next posedge, compare C. Also assert that when reset high on posedge, C==0.

ALU_OVERFLOW_SIGN	Observe signed overflow/interesting sign-extension behavior for results outside 5-bit signed range (e.g., 7+7=14). Ensure C holds correct 5-bit two's complement result.	Directed: vectors that cause overflow (7+7, -8 + -8, etc.). Also randomize.	Coverpoints: result sign (positive/negative), overflow-inducing pairs as bins. Cross Opcode with overflow bin.	Checker: compute <code>expected = (a op b)</code> with signed arithmetic truncated/extended to 5 bits as in RTL; compare to C. Report overflow cases and mismatches.
ALU_OPCODE_EXHAUST	All legal opcodes (ADD, SUB, NOT_A, REDUCTION_OR) must be exercised.	Constrained random with explicit constraints that all opcode values appear; plus directed sequences that force each opcode at least once.	Coverpoint: Opcode with 4 bins. Goal: 100% (each opcode hit). Cross Opcode with operation-specific A/B bins.	Checker: golden-model uses Opcode to calculate expected; mismatch tracking per-opcode.
ALU_BOUNDARY_VALUES	Test A/B at boundary values: +7, -8, 0, +1, -1 to ensure edge-case correctness.	Directed + constrained random with bias toward boundary values (use dist or explicit add).	Coverpoints: A and B bins with those exact values. Cross with Opcode.	Checker: as above — compare expected results for each vector; log if any boundary mismatch.
ALU_DEFAULT_BRANCH_JUSTIFY	default branch in combinational case sets <code>Alu_out = 0</code> — explain coverage expectations.	<i>Comment:</i> Because Opcode is a typed enum with only 4 legal values, default is defensive and unreachable with legal stimulus. If required, one can create a directed illegal-value injection using a bit-level poke or *Casting (but this changes type discipline).	Coverage: mark default branch as <i>not covered</i> and justify in report (defensive code). If grader requires 100% code coverage including default, plan a single directed <i>illegal opcode</i> injection (explain how and why).	Checker: If illegal opcode injected, golden model uses <code>default => expected=0</code> . Testbench will detect match/mismatch. If no illegal injection, document justification for not covering default.

6. Do file

```

1  vlib work
2  vlog ALU.v ALU_pkg.svh ALU_tb.svh +cover -covercells
3  vsim -voptargs=+acc work.ALU_tb -cover
4  add wave *
5  coverage save ALU_tb.ucdb -onexit -du work.ALU
6  run -all

```

7. Code Coverage report snippets

code_cvr_alu - Notepad

File Edit Format View Help

Coverage Report by instance with details

```
=====
=== Instance: /\ALU_tb#dut
=== Design Unit: work.ALU
=====
```

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	2	2	0	100.00%

```
=====Branch Details=====
```

Branch Coverage for instance /\ALU_tb#dut

Line	Item	Count	Source
----	----	----	-----
File ALU.v			
-----IF Branch-----			
36		211	Count coming in to IF
36	1	29	if (reset)
38	1	182	else

Branch totals: 2 hits of 2 branches = 100.00%

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	8	8	0	100.00%

```
=====Statement Details=====
```

Statement Coverage for instance /\ALU_tb#dut --

Line	Item	Count	Source
----	----	----	-----
File ALU.v			
1			module ALU (
2			input clk,

Statements	8	8	0	100.00%
=====Statement Details=====				
Statement Coverage for instance /\ALU_tb#dut --				
Line	Item	Count	Source	
----	----	-----	-----	
File ALU.v				
1			module ALU (	
2			input clk,	
3			input reset,	
4			input [1:0] Opcode, // The opcode	
5			input signed [3:0] A, // Input data A in 2's complement	
6			input signed [3:0] B, // Input data B in 2's complement	
7				
8			output reg signed [4:0] C // ALU output in 2's complement	
9				
10			);	
11				
12			reg signed [4:0]	Alu_out; // ALU output in 2's complement
13				
14			localparam	Add = 2'b00; // A + B
15			localparam	Sub = 2'b01; // A - B
16			localparam	Not_A = 2'b10; // ~A
17			localparam	ReductionOR_B = 2'b11; // B

```

18
19                                // Do the operation
20                                //covered all cases
21            1                    201    always @* begin
22                                case (Opcode)
23            1                    41        Add:            Alu_out = A + B;
24            1                    52        Sub:            Alu_out = A - B;
25            1                    49        Not_A:          Alu_out = ~A;
26            1                    59        ReductionOR_B:  Alu_out = |B;
27                                endcase
28                                end // always @ *
29
30                                // Register output C
31                                // why do not remove c ??
32                                // why use internal signal alu_out!!!?
33                                //maybe to avoid latches???!!
34                                //you want alu with register?
35            1                    211    always @(posedge clk or posedge reset) begin
36                                if (reset)
37            1                    29        C <= 5'b0;
38                                else

```

code_cvr_alu - Notepad

File Edit Format View Help

```
34                                //you want alu with register?
35                                1          211      always @(posedge clk or posedge reset) begin
36                                if (reset)
37                                1          29          C <= 5'b0;
38                                else
39                                1          182         C<= Alu_out;
```

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	44	44	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /\ALU_tb#dut --

Node	1H->0L	0L->1H	"Coverage"
A[0-3]	1	1	100.00
Alu_out[4-0]	1	1	100.00
B[0-3]	1	1	100.00
C[4-0]	1	1	100.00
Opcode[0-1]	1	1	100.00
clk	1	1	100.00
reset	1	1	100.00

Total Node Count = 22

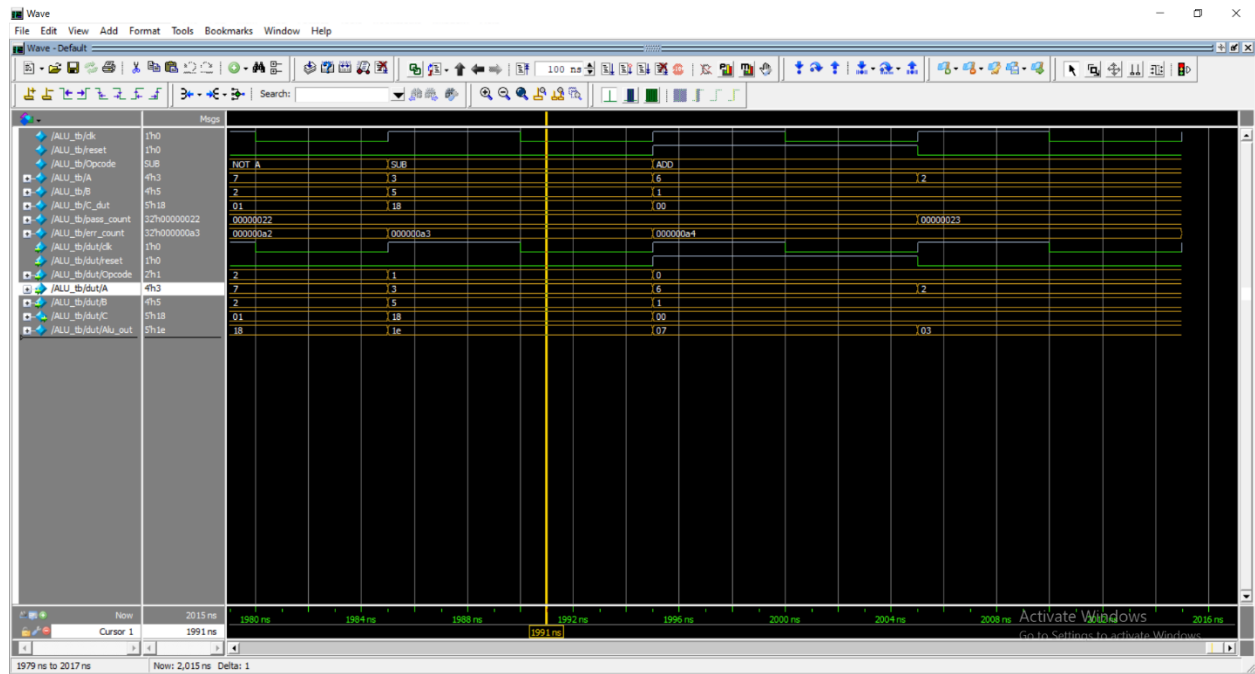
Toggled Node Count = 22

Untoggled Node Count = 0

Toggle Coverage = 100.00% (44 of 44 bins)

Total Coverage By Instance (filtered view): 100.00%

8. Clear and neat QuestaSim waveform snippets showing the functionality of the design



Question 3

1. Testbench code

```
1  module FSM_tb;
2
3      import fsm_pkg::*;
4
5      // DUT signals
6      logic clk, rst, x, y;
7      logic [9:0] users_count;
8
9      // Instantiate DUT
10     FSM_010 dut (.clk(clk), .rst(rst), .x(x), .y(y), .users_count(users_count));
11
12     // Clock generation
13     initial clk = 0;
14     always #5 clk = ~clk;
15
16     // Transaction object
17     fsm_transaction tr;
18
19     // Golden model states
20     state_e cs, ns;
21
22     // Stimulus and checking
23     initial begin
24         tr = new();
25
26         // Reset sequence
27         rst = 1;
28         #15 rst = 0;
29
30         repeat (20000) begin
31             assert(tr.randomize()); // generate random input
32             x = tr.x;
33             rst = tr.rst;
34
```

```

35     @(posedge clk);
36
37     check_result(tr);
38 end
39
40 $display("TEST COMPLETED");
41 $stop;
42 end
43
44 // Check task
45 task check_result(fsm_transaction tr);
46     golden_model(tr);
47
48     // Compare expected vs DUT
49     if (tr.y_exp != y || tr.user_count_exp != users_count) begin
50         $error("Mismatch! Expected y=%0d, count=%0d | Got y=%0d, count=%0d",
51             tr.y_exp, tr.user_count_exp, y, users_count);
52     end
53 endtask
54
55 // Golden model task
56
57 task golden_model(fsm_transaction tr);
58     static bit [9:0] exp_count = 0;
59
60     if (tr.rst) begin
61         cs = IDLE;
62         exp_count = 0;
63         tr.y_exp = 0;
64         tr.user_count_exp = 0;
65     end else begin
66         // next state

```



```

66 // next state
67 case (cs)
68     IDLE: ns = (tr.x==0) ? ZERO : IDLE;
69     ZERO: ns = (tr.x==1) ? ONE : ZERO;
70     ONE: ns = (tr.x==0) ? STORE : IDLE;
71     STORE: ns = (tr.x==0) ? ZERO : IDLE;
72 endcase
73
74
75 if (ns == STORE) begin
76     exp_count++;
77     tr.y_exp = 1;
78 end else begin
79     tr.y_exp = 0;
80 end
81 tr.user_count_exp = exp_count;
82 cs = ns;
83 end
84 endtask
85
86
87
88 endmodule

```

2. Package code

```

1 package fsm_pkg;
2
3 // FSM states typedef
4 typedef enum logic [1:0] {
5     IDLE   = 2'b00,
6     ZERO   = 2'b01,
7     ONE    = 2'b10,
8     STORE  = 2'b11
9 } state_e;
10
11 // Transaction class
12 class fsm_transaction;
13     rand bit x;
14     rand bit rst;
15     bit y_exp;
16     bit [9:0] user_count_exp;
17
18     // Constraints
19     constraint rst_c { rst dist {1 := 1, 0 := 9}; } // reset mostly inactive
20     constraint x_c   { x dist {0 := 67, 1 := 33}; } // x=0 ~67%
21
22 endclass
23
24 endpackage
25

```

3. Design code

```

1  //////////////////////////////////////
2  // Author: Kareem Waseem
3  // Course: Digital Verification using SV & UVM
4  //
5  // Description: 010-sequence-detector Design
6  //
7  //////////////////////////////////////
8  module FSM_010(clk, rst, x, y, users_count);
9      parameter IDLE = 2'b00;
10     parameter ZERO = 2'b01;
11     parameter ONE = 2'b10;
12     parameter STORE = 2'b11;
13
14     input clk, rst, x;
15     output y;
16     output reg [9:0] users_count;
17
18     reg [1:0] cs, ns;
19
20     always @(*) begin
21         case (cs)
22             IDLE:
23                 if (x)
24                     ns = IDLE;
25                 else
26                     ns = ZERO;
27             ZERO:
28                 if (x)
29                     ns = ONE;
30                 else
31                     ns = ZERO;
32             ONE:
33                 if (x)
34                     ns = IDLE;
35                 else
36                     ns = STORE;

```

```

37         STORE:
38             if (x)
39                 ns = IDLE;
40             else
41                 ns = ZERO;
42             default:    ns = IDLE;
43         endcase
44     end
45
46     always @(posedge clk or posedge rst) begin
47         if(rst) begin
48             cs <= IDLE;
49         end
50         else begin
51             cs <= ns;
52         end
53     end
54
55     always @(posedge clk or posedge rst) begin
56         if(rst) begin
57             users_count <= 0;
58         end else begin
59             if (ns == STORE) // FIXED: check ns, not cs
60                 users_count <= users_count + 1;
61         end
62     end
63
64     assign y = (ns == STORE);
65
66 endmodule

```

4. Report any bugs detected in the design and fix them

Fixed to depend on next state and input

5. Snippet to your verification plan document

Label	Design Requirement	Stimulus Generation	Functional Coverage	Functionality Check
FSM_1	When reset is asserted, FSM goes to IDLE, y=0, users_count=0	Directed (reset at sim start, random during sim)	100% reset scenarios	Checker ensures outputs correct
FSM_2	FSM detects "010" → y=1 at STORE, users_count++	Randomized x stream	Cover transitions: IDLE→ZERO→ONE→STORE	Checker verifies increment & y pulse
FSM_3	FSM ignores overlapping patterns (non-overlapped detection)	Randomized long streams with patterns	Cover "010010", "0101"	Checker ensures only non-overlap detection
FSM_4	FSM loops correctly on non-matching sequences	Randomized x=0 (67%) vs x=1 (33%)	Cover IDLE→ZERO, ZERO→ZERO, ONE→IDLE	Checker compares with golden model

6. Do file

```

1  vlib work
2  vlog FSM_010.v FSM_pkg.svh FSM_tb.svh +cover -covercells
3  vsim -voptargs=+acc work.FSM_tb -cover
4  add wave *
5  coverage save FSM_tb.ucdb -onexit -du work.FSM_010
6  run -all

```

7. Code Coverage report snippets

Coverage Report by instance with details

```

=====
=== Instance: /\FSM_tb#dut
=== Design Unit: work.FSM_010
=====

```

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	18	18	0	100.00%

=====Branch Details=====

Branch Coverage for instance /\FSM_tb#dut

Line	Item	Count	Source
----	----	----	-----
File FSM_010.v			
-----CASE Branch-----			
21		19816	Count coming in to CASE
22	1	5761	IDLE:
27	1	7703	ZERO:
32	1	3965	ONE:
37	1	2387	STORE:

Branch totals: 4 hits of 4 branches = 100.00%

-----IF Branch-----			
23		5761	Count coming in to IF
23	1	2410	if (x)
25	1	3351	else

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----			
28		7703	Count coming in to IF
28	1	3245	if (x)
30	1	4458	else

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
 33                               3965    Count coming in to IF
 33             1                1524
                               1524    if (x)

 35             1                2441
                               2441    else
```

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
 38                               2387    Count coming in to IF
 38             1                913
                               913    if (x)

 40             1                1474
                               1474    else
```

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
 47                               17373   Count coming in to IF
 47             1                3515   if(rst) begin

 50             1                13858   else begin
```

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
 56                               14489   Count coming in to IF
 56             1                3515   if(rst) begin

 58             1                10974   end else begin
```

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
 59                               10974   Count coming in to IF
 59             1                1842   if (ns == STORE) // FIXED: check ns, not cs

                               9132   All False Count
```

Branch totals: 2 hits of 2 branches = 100.00%

Condition Coverage:

Enabled Coverage	Bins	Covered	Misses	Coverage
Conditions	1	1	0	100.00%

====Condition Details=====

Condition Coverage for instance /\FSM_tb#dut --

File FSM_010.v

-----Focused Condition View-----

Line 59 Item 1 (ns == 3)

Condition totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
(ns == 3)	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	(ns == 3)_0	-
Row 2:	1	(ns == 3)_1	-

Expression Coverage:

Enabled Coverage	Bins	Covered	Misses	Coverage
Expressions	1	1	0	100.00%

====Expression Details=====

Expression Coverage for instance /\FSM_tb#dut --

File FSM_010.v

-----Focused Expression View-----

Line 64 Item 1 (ns == 3)

Expression totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
(ns == 3)	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
ROW 1:	1	(ns == 3)_0	-
ROW 2:	1	(ns == 3)_1	-

FSM Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
FSM States	4	4	0	100.00%
FSM Transitions	7	7	0	100.00%

=====FSM Details=====

FSM Coverage for instance /\FSM_tb#dut --

FSM_ID: cs

Current State Object : cs

State Value MapInfo :

Line	State Name	Value
22	IDLE	0
27	ZERO	1
32	ONE	2
37	STORE	3

Covered States :

State	Hit_count
IDLE	5802
ZERO	5844
ONE	2758
STORE	1673

Covered Transitions :

Line	Trans_ID	Hit_count	Transition
26	0	2647	IDLE -> ZERO
29	1	2758	ZERO -> ONE

code_cvr_fsm - Notepad

File Edit Format View Help

29	1	2758	ZERO -> ONE
48	2	905	ZERO -> IDLE
36	3	1673	ONE -> STORE
34	4	1085	ONE -> IDLE
41	5	1016	STORE -> ZERO
39	6	656	STORE -> IDLE

Summary	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
FSM States	4	4	0	100.00%
FSM Transitions	7	7	0	100.00%

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	16	16	0	100.00%

=====Statement Details=====

Statement Coverage for instance /\FSM_tb#dut --

Line	Item	Count	Source
----	----	----	-----
File FSM_010.v			
8			module FSM_010(clk, rst, x, y, users_count);
9			parameter IDLE = 2'b00;
10			parameter ZERO = 2'b01;
11			parameter ONE = 2'b10;
12			parameter STORE = 2'b11;
13			
14			input clk, rst, x;
15			output y;
16			output reg [9:0] users_count;
17			

17

18

reg [1:0] cs, ns;

19

20

1

19816

always @(*) begin

21

case (cs)

22

IDLE:

23

if (x)

24

1

2410

ns = IDLE;

25

else

26

1

3351

ns = ZERO;

27

ZERO:

28

if (x)

29

1

3245

ns = ONE;

30

else

31

1

4458

ns = ZERO;

32

ONE:

33

if (x)

34

1

1524

ns = IDLE;

35

else

36

1

2441

ns = STORE;

37

STORE:

38

if (x)

code_cvr_fsm - Notepad

File Edit Format View Help

```
38                                     if (x)
39             1                      913                                ns = IDLE;
40                                     else
41             1                      1474                               ns = ZERO;
42                                     default:      ns = IDLE;
43                                     endcase
44                                     end
45
46             1                      17373      always @(posedge clk or posedge rst) begin
47                                     if(rst) begin
48             1                      3515                                cs <= IDLE;
49                                     end
50                                     else begin
51             1                      13858                                cs <= ns;
52                                     end
53                                     end
54
55             1                      14489      always @(posedge clk or posedge rst) begin
56                                     if(rst) begin
57             1                      3515                                users_count <= 0;
58                                     end else begin
59                                     if (ns == STORE) // FIXED: check ns, not cs
```

```

code_cvr_fsm - Notepad
File Edit Format View Help

59                                     if (ns == STORE) // FIXED: check ns, not cs
60             1                      1842          users_count <= users_count + 1;
61                                     end
62                                     end
63
64             1                      14936          assign y = (ns == STORE);

Toggle Coverage:
  Enabled Coverage      Bins      Hits      Misses      Coverage
  -----
  Toggles              24        24         0      100.00%

=====Toggle Details=====

Toggle Coverage for instance /\FSM_tb#dut  --

      Node      1H->0L      0L->1H      "Coverage"
      -----
      clk          1          1          100.00
      cs[1-0]       1          1          100.00
      ns[1-0]       1          1          100.00
      rst          1          1          100.00
      users_count[3-0] 1          1          100.00
      x            1          1          100.00
      y            1          1          100.00

Total Node Count      =      12
Toggled Node Count    =      12
Untoggled Node Count  =       0

Toggle Coverage       =      100.00% (24 of 24 bins)

Total Coverage By Instance (filtered view): 100.00%

```

8. Clear and neat QuestaSim waveform snippets showing the functionality of the design

